

Beam Element Section Property Tool - Final Report

Subtitle: Functional engineering software deliverable for beam cross-section property calculation
Repository: <https://github.com/BESPsoftware/beam-section-property-tool> Technology stack: C++17, CMake, C-compatible shared library, optional Qt Widgets GUI, CTest, GitHub Actions Final report date: 2026-06-08 Deliverable status: Core/API implementation complete for the documented scope; external/manual validation items identified separately Submission context: Academic/professional software project submission Team/course: Not specified in the repository package

1. Executive Summary

Beam Element Section Property Tool is a C++17 engineering software project for modelling beam cross-sections and calculating section properties used in structural and finite-element workflows. It supports predefined parametric sections, user-defined thin-walled plate sections, stress output locations, lightweight mesh visualization, and result export for downstream review.

The project deliverable includes a reusable calculation core, a C-compatible shared-library API, an optional Qt Widgets GUI, example programs, CMake build configuration, Windows build/deployment support, automated tests, numerical validation material, and user/developer documentation. The same calculation and data-model layers are used by the API and GUI, so interactive and programmatic workflows are built on the same core implementation.

The verified local no-GUI build performed for this report completed successfully. CMake configured the core/API build, the project built successfully, CTest executed the aggregated core/API regression target successfully, all three example programs ran, and git diff --check reported no whitespace issues.

Remaining work is bounded as validation and hardening rather than missing core implementation: manual Windows Qt deployment and visual inspection, solver-side acceptance of ANSYS/ABAQUS/Midas Civil outputs, confirmation or formal sign-off of arbitrary non-reference crane girder construction rules, final DOCX manual review, and small documentation maintenance tasks.

2. Project Scope and Objectives

The tool addresses a common engineering software need: computing cross-section properties of beam elements from consistent geometric inputs. These properties support structural review, stress recovery workflows, and finite-element model preparation. The project focuses on reproducible section-property calculation and integration surfaces rather than a full structural solver.

The intended users are:

- Engineers and evaluators who need section properties and stress output points for common beam section geometries.
- Users who need to define thin-walled plate sections interactively through a Canvas workflow.
- Developers or automation scripts that need a C-compatible shared-library API.
- Downstream FEM users who need generated section/property data for solver review.

The expected workflow is:

- Select a predefined section type or define a Canvas thin-walled section.
- Enter dimensions in millimeters.
- Validate and calculate the section model.
- Review area, inertias, centroid, principal axes, shear areas, torsion-related values, warping constant, and shear center.
- Inspect or edit stress output points.
- Review a lightweight visualization mesh.
- Export results through CSV, JSON, or FEM-oriented text formats.

The supported section types are:

- H section (SPT_H_SECTION)
- Hollow box section (SPT_BOX_SECTION)
- Pipe section (SPT_PIPE_SECTION)
- Quayside crane girder (SPT_CRANE_GIRDER)
- Canvas thin-walled section (SPT_CANVAS)

The primary outputs are Area, Jz, Jy, Jyz, Jzo, Jyo, Jx, Az, Ay, cy, cz, theta, Cw, ys, and zs, plus stress point coordinates, mesh counts, and export files.

3. Assignment Coverage

Required area	Repository evidence	Coverage statement	Status
Requirements and design documents	Source requirement PDF/workbook, architecture notes, implementation status, README, developer guide, tests and validation fixtures	Requirements are represented through source materials, reference data, design notes, and executable evidence rather than a single monolithic SRS.	Available and evidence-based
Development artifacts	Source/, Examples/, CMakeLists.txt, build_windows.bat, GitHub Actions workflow	The calculation core, shared API, optional GUI, exporters, examples, build targets, and platform build support are implemented for the documented scope.	Implemented
Testing and quality assurance	CTest target, regression tests, API integration tests, Qt smoke target, numerical validation report, acceptance checklist, defect log, CI workflow	Core/API tests and examples pass in the local no-GUI verification; GUI smoke and Windows deployment validation are documented separately.	Implemented and verified
User documentation	User manual, API reference, developer guide, Windows Qt verification guide, README, DOCX user manual package	Build, usage, API, GUI workflow, export behavior, and troubleshooting guidance are available; DOCX final formatting review remains a manual packaging step.	Available, final manual review pending
External/manual validation	Windows Qt verification checklist, FEM solver acceptance notes, final acceptance checklist	Manual Windows deployment and solver-side acceptance are identified as external validation tasks beyond local core/API verification.	Available, external validation pending

4. Functional Requirements and Implemented Capabilities

The software implements section creation, validation, calculation, visualization, API integration, and export workflows for the documented beam-section scope.

Supported Geometry Inputs

The predefined parametric section inputs use named dimensions. The API and GUI use the same parameter names:

- H section: A, H, e, f
- Box section: A, B, H, D, E, H1, D1, E1
- Pipe section: Do, t
- Quayside crane girder: A, B, G, D, e, f, H, W, M, N, p, s, t, u, M1, k, k1, h, h1
- Canvas section: plate centerline start/end coordinates, plate thickness, material id, and optional plate id

Canvas sections allow arbitrary user-defined thin-walled plate layouts. Plates are modelled by centerline endpoints and thickness; the implementation creates plate polygons and a rectangularized component representation for property and mesh workflows.

Input Validation

The builder validates required parameters and rejects invalid inputs before calculation. Examples include:

- Required parametric dimensions must be positive.
- H-section web thickness must not exceed flange width.

- Box-section outer width and side web geometry must leave valid clearances.
- Pipe thickness must be less than half the outer diameter.
- Canvas plates must have positive thickness and nonzero length.
- Explicit Canvas plate identifiers must be unique.

Invalid input produces diagnostics rather than silently continuing.

Calculated Results

The property result structure contains:

- Cross-sectional area.
- Second moments about global axes.
- Product of inertia.
- Principal second moments.
- Torsion-related constant used by the tool.
- Shear areas.
- Centroid coordinates.
- Principal-axis rotation.
- Warping constant.
- Shear-center coordinates.

Stress output points are generated for each section type. They include global y/z coordinates and principal y0/z0 coordinates. The stress point engine supports coordinate transforms and validation against the current section geometry.

Mesh and Visualization

The mesh engine creates lightweight triangular visualization meshes. Rectangular and plate-based sections are meshed through rectangular components; pipe sections are meshed as annular triangular subdivisions. The mesh is intended for display and summary counts, not as a solver-quality FEM mesh.

Export and Integration

The exporter supports:

- CSV property and stress point output.
- JSON property, stress point, and mesh summary output.
- ANSYS general beam-section card output.
- ABAQUS beam general section output.
- Midas Civil user-defined section output.

The C-compatible API exposes these workflows for use from C, C++, and foreign-function interfaces.

5. Design Requirements, Assumptions, and Constraints

All geometric inputs are interpreted in millimeters. Area and shear area are reported in square millimeters, inertias and torsion-related values in millimeters to the fourth power, warping constant in millimeters to the sixth power, coordinates in millimeters, and principal-axis angle in radians.

The project uses double precision internally. GUI tables may display rounded values for readability, but calculations use the core numeric representation.

The source requirement package contains a PDF and an Excel workbook. The workbook data is represented in `tests/fixtures/test_data_reference.json` and is used as numerical reference evidence for H, hollow box, pipe, and the supplied crane girder reference case. The source PDF diagram is also the basis for crane girder geometry labels, but the repository documentation and implementation note that it does not fully define every construction coordinate for arbitrary crane girder inputs.

The main modelling assumptions are:

- H, box, and pipe reference cases use analytical formulas aligned to the supplied workbook.
- The pipe shear-area value follows the workbook factor rather than a pure half-area idealization.
- Canvas sections use thin-walled plate centerlines converted to plate polygons and rectangularized property components.
- Open-section torsion for plate-based sections uses a plate-length and thickness expression.
- H-section warping constant uses the documented equal-flange formula.
- Symmetric closed box and pipe sections use zero warping constant as the exact closed-section result represented by the implementation.
- Crane girder and Canvas shear-center/warping behavior uses numerical open thin-walled sectorial-area integration.
- The exact crane girder workbook case is treated as authoritative; non-reference crane girder inputs use the available named plate graph and emit an approximate-geometry diagnostic.

The regression tolerances documented by the project are:

- Coordinates, area, and shear area: $\max(0.01, 1e-6 * \text{abs}(\text{expected}))$
- Inertia and torsion: $\max(1.0, 1e-6 * \text{abs}(\text{expected}))$
- Principal-axis angle: $1e-8$ rad

These tolerances are encoded in the regression test suite and summarized in the numerical validation report.

6. System Architecture and Design Rationale

The architecture is organized around a Qt-independent core with optional user interface and integration layers. This separation allows the core library, API, tests, and examples to build without Qt, while still supporting a GUI when Qt Widgets is available.

The main modules are:

- Source/common: shared data model, section types, property structures, stress point structures, mesh structures, materials, and diagnostics.
- Source/geometry: section builders, parametric geometry construction, Canvas plate validation, plate polygons, contours, and rectangle components.
- Source/calculation: area, centroid, inertia, torsion-related quantities, shear areas, principal axes, warping constant, and shear-center calculations.
- Source/stress: default stress point generation, coordinate transformations, and stress point validation.
- Source/mesh: lightweight triangular visualization mesh generation.
- Source/import_export: CSV, JSON, ANSYS, ABAQUS, and Midas Civil writers.
- Source/api: C ABI wrapper and exported shared-library interface.
- Source/gui: optional Qt Widgets application.

The data flow is:

```
GUI/API input
-> SectionParameters or PlateSegment[]
-> SectionBuilder validation and SectionModel construction
-> SectionCalculator property calculation
-> StressPointEngine stress output point generation
-> MeshEngine visualization mesh generation
-> GUI display, API return values, or Exporter output files
```

This architecture keeps the computational core reusable and testable. The API and GUI are clients of the same core data model. The C API presents stable plain-data boundaries; the GUI presents interactive workflows over the same underlying operations.

7. Implementation Details and Development Artifacts

The implementation is concentrated in a small number of focused modules.

Source/common/DataModel.h defines the central data structures: section type enums, section

parameters, materials, rectangles, plate segments, contours, properties, stress points, mesh data, diagnostics, section models, and calculation results. This common layer allows builders, calculators, exporters, tests, API code, and GUI code to exchange the same objects.

Source/geometry/SectionBuilder.cpp is responsible for constructing valid section models. It builds H, box, pipe, crane girder, and Canvas sections. For H and box sections it assembles rectangular components and contours. For pipe sections it creates circular contour data. For Canvas sections it validates plate data, generates automatic plate identifiers where needed, converts plate centerlines to polygons, and stores the resulting plates and components. For the crane girder it creates a named plate graph based on the available requirement diagram labels and stores workbook-derived override values for the exact reference case.

Source/calculation/SectionCalculator.cpp performs the section property calculations. It implements analytical formulas for supported simple sections, composite rectangle calculations for plate-based sections, principal axes from the inertia tensor, torsion-related values, shear areas, warping constant, and shear-center behavior. Its open thin-walled sectorial-area integration supports crane and Canvas warping/shear-center calculations.

Source/stress/StressPointEngine.cpp creates default stress output locations and transforms coordinates between global and principal systems. For H, box, and pipe sections it provides section-specific default points. For crane and Canvas sections it uses plate endpoints for default output points. It can also validate whether a stress point lies on or outside the modelled section.

Source/mesh/MeshEngine.cpp generates visualization mesh data. Rectangular components are divided into triangular cells according to target size or refinement factor. Pipe sections are triangulated as annular subdivisions. The mesh model stores nodes, triangles, boundary edges, and diagnostics.

Source/import_export/Exporter.cpp writes output files. CSV and JSON exports provide property and stress-point data. ANSYS, ABAQUS, and Midas Civil writers emit general section/property cards or text sections for downstream solver review. The Windows path logic uses UTF-8 to wide-character conversion so exported files can be written to Unicode paths on Windows.

Source/api/section_property_tool.cpp wraps the core in a C-compatible interface. It converts API enums and structs to core types, manages opaque handles, allocates/free stress point arrays, exposes mesh counts, routes exports, and provides thread-local error reporting.

Source/gui/main.cpp implements the optional Qt Widgets GUI. It includes the main cross-section window, parameter and property tables, graphics views, stress point editing, mesh visualization, and the Canvas drawing/editing workflow.

Examples/ contains three buildable programs:

- example1_parametric: creates an H section and prints area, inertia, warping constant, and shear center.
- example2_canvas: creates a three-plate Canvas section and prints area and centroid.
- example3_dll_batch: runs multiple API calculations in a batch-style workflow.

CMakeLists.txt builds the static core library, shared API library, tests, examples, optional GUI, and optional GUI smoke test. build_windows.bat supports Windows Qt build/deployment workflows with MSVC or MinGW depending on installed Qt paths.

8. Engineering Calculations and Validation Approach

The calculation strategy combines analytical formulas, composite geometry calculations, and workbook-based reference validation.

For H sections, the geometry is represented by top flange, web, and bottom flange rectangles. The tool calculates area, centroid, global inertias, shear areas, torsion-related constant, principal axes, and an analytical warping constant. The reference H-section case uses $A = 100 \text{ mm}$, $H = 210 \text{ mm}$, $e = 20 \text{ mm}$, and $f = 12 \text{ mm}$; the expected area is 6520 mm^2 .

For hollow box sections, the builder constructs bottom plate, top plate, and side web rectangles

based on the documented eight parameters. The calculation includes workbook-aligned area, centroid, inertias, torsion-related constant, and shear areas. The reference case uses dimensions including $A = 1320$ mm, $B = 1250$ mm, $H = 2600$ mm, $D = 40$ mm, $E = 16$ mm, $H1 = 600$ mm, $D1 = 22$ mm, and $E1 = 16$ mm; the expected area is 165040 mm².

For pipe sections, the model uses outer diameter D_o and thickness t . The implementation calculates annular area and moments analytically. The reference case uses $D_o = 1300$ mm and $t = 14$ mm; the expected area is approximately 56561.234135 mm². The documented shear-area value follows a workbook-specific factor slightly above one half of area.

For the quayside crane girder, the reference parameter set is recovered from the workbook and treated as the numerical acceptance case. The implementation stores the reference properties and stress points for that exact case because the requirement diagram does not fully define arbitrary coordinate construction rules. Other crane parameter sets are built through the available named plate graph and reported as approximate.

For Canvas thin-walled sections, users provide plate centerlines and thicknesses. The software converts those plates into geometry suitable for property and mesh calculations. The documented example has three plates:

- bottom: (0, 0) to (100, 0), thickness 10
- web: (50, 0) to (50, 120), thickness 8
- top: (0, 120) to (100, 120), thickness 10

That example reports area 2960 mm² and centroid (50, 60) mm.

Principal properties are derived from the centroidal inertia tensor. The documented formula for principal-axis angle is:

$$\theta = 0.5 \cdot \text{atan2}(-2 \cdot J_{yz}, J_y - J_z)$$

The principal inertias are the eigenvalues of the centroidal inertia tensor.

The numerical validation report records workbook expected values against computed values. The summary is:

Section	Property checks	Stress point checks	Result
H section	8/8	4/4	PASS
Hollow box	8/8	4/4	PASS
Pipe	8/8	4/4	PASS
Crane girder reference case	13/13	4/4	PASS

Across the workbook-backed validation set, the report records 41 numerical checks passing within the encoded tolerances. Additional regression checks cover shear-center and warping-constant behavior. For H sections, the shear center matches the centroid and the warping constant uses the analytical equal-flange formula. For box and pipe sections, the shear center matches the centroid and warping constant is zero for the symmetric closed section representation. For crane and Canvas sections, the implementation uses numerical open thin-walled sectorial-area integration and regression checks expected qualitative behavior such as magnitude, sign, and nonzero offsets.

9. API and Integration Interface

The public integration interface is a C-compatible API declared in Source/api/section_property_tool.h. It is designed for C, C++, and foreign-function interfaces.

The API uses opaque handles:

- SptSectionHandle for built section models.
- SptResultHandle for calculated results.
- SptMeshHandle for generated mesh data.

Each handle has an explicit destroy function. Stress point arrays returned from the API are explicitly released with `spt_free_stress_point_array`. This avoids exposing C++ object ownership across the ABI boundary.

The API lifecycle is:

```
spt_create_section_from_parameters(...) or spt_create_section_from_canvas_lines(...)
spt_calculate_section_properties(...)
spt_get_result_properties(...)
spt_get_result_stress_points(...) or spt_create_mesh(...)
spt_export_results(...)
spt_destroy_result(...)
spt_destroy_section(...)
```

All API functions return 0 on success. Nonzero return values indicate failure, and the caller can inspect `spt_get_last_error()`. The error structure includes code, severity, field, message, and remediation fields. The last error is thread-local in the implementation.

A minimal H-section API workflow is:

```
const SptParameter values[] = {
    {"A", 100.0},
    {"H", 210.0},
    {"e", 20.0},
    {"f", 12.0},
};

SptSectionParameters params = {SPT_H_SECTION, values, 4};
SptSectionHandle section = NULL;
spt_create_section_from_parameters(&params, &section);

SptResultHandle result = NULL;
spt_calculate_section_properties(section, &result);

SptSectionProperties props;
spt_get_result_properties(result, &props);

spt_export_results(result, "section.csv", SPT_EXPORT_CSV);

spt_destroy_result(result);
spt_destroy_section(section);
```

The shared-library target is `SectionPropertyApi` and the output name is `SectionPropertyTool`. On macOS/Linux the artifact is a `.dylib` or `.so`; on Windows the documented artifacts are `SectionPropertyTool.dll` and `SectionPropertyTool.lib`.

10. Graphical User Interface and User Workflow

The GUI is an optional Qt Widgets application. It is not required for the core/API build, but it provides an interactive workflow when Qt is installed.

The main window is organized into four tabs:

- General
- Stress Points
- FE Mesh
- Canvas

The General tab allows users to choose a section type, edit geometry parameters, apply calculations, review the section drawing, and inspect calculated properties. The table displays area, inertias, product of inertia, principal inertias, torsion-related constant, shear areas, centroid, principal-axis angle, warping constant, and shear-center coordinates.

The Stress Points tab lists stress output locations with IDs, global coordinates, and principal coordinates. Editing global y or z updates the principal coordinates and redraws markers in the graphics view. Reset Defaults recalculates the generated stress points.

The FE Mesh tab displays the lightweight triangular visualization mesh generated from the current section. A refinement factor requests denser or coarser display meshes. This mesh is for visualization and summary counts, not solver-quality export.

The Canvas tab provides advanced graphical input for user-defined thin-walled sections. Users can add rows manually, draw plate centerlines, set plate thickness, assign or auto-generate plate IDs, delete selected plates, clear the Canvas, and build the Canvas section. Selection stays synchronized between the plate table and drawing. Select/Edit mode supports endpoint dragging with table write-back on release. The view supports grid display and snap-to-grid for drawing and editing.

The General, Stress Points, FE Mesh, and Canvas graphics views support zoom, mouse-wheel zoom, panning, fit-to-scene, and reset. After successful parametric recalculation or Canvas build, the GUI refreshes properties, stress points, mesh, and previews from the active section geometry.

11. Export and Downstream Workflow Support

Export is available through the shared-library API and implemented in the core exporter module.

CSV output contains a property table and stress point rows. JSON output contains properties, stress points, and a mesh summary. These formats are useful for spreadsheet review, scripting, and downstream data exchange.

ANSYS export writes a Mechanical APDL beam-section card using a general arbitrary section representation. The writer emits SECTYPE, SECOFFSET,BSEC, and SECDATA content. The SECOFFSET,BSEC line is specifically included to avoid double-counting centroid offsets in the generated ASEC workflow.

ABAQUS export writes *BEAM GENERAL SECTION, SECTION=GENERAL content with area, inertias, product of inertia, torsion-related constant, and section point output when stress points are available.

Midas Civil export writes a Midas Civil Text user-defined section with area, shear areas, torsion-related constant, and inertias.

The API integration tests check generated content for ANSYS, ABAQUS, and Midas Civil, including empty-stress-point behavior. Solver-side acceptance in the target applications is still an external validation step because it requires running the generated cards inside those commercial tools.

12. Testing and Quality Assurance

The project uses an aggregated CTest target for core/API coverage plus optional Qt GUI smoke testing when Qt test components are available.

The core/API test executable is SectionPropertyTests. It is built from:

- tests/unit/test_main.cpp
- tests/regression/test_regression.cpp
- tests/integration/test_api.cpp

The test runner executes regression and API integration checks. The regression suite covers:

- H-section workbook values and four stress points.
- Hollow box workbook values and four stress points.
- Pipe workbook values and four stress points.
- Supplied crane girder workbook values, principal properties, and four stress points.
- Warping constant and shear-center behavior.
- Non-reference crane girder approximate-path diagnostics and deterministic behavior.
- Invalid Canvas inputs, duplicate IDs, auto-generated IDs, and valid Canvas calculations.

The API integration tests cover:

- API version reporting.
- Section creation.

- Property calculation.
- Property retrieval including warping constant and shear-center fields.
- Stress point retrieval and memory cleanup.
- Mesh creation and mesh counts.
- CSV export.
- ANSYS, ABAQUS, and Midas Civil export content.
- Empty stress-point export paths.
- UTF-8 export paths.

The optional GUI smoke test target is SectionPropertySmokeTest. When Qt Widgets and Qt Test are available, it constructs the GUI using the offscreen Qt platform, switches through the parametric section types, checks property and stress table behavior, verifies mesh scene content, and builds the documented three-plate Canvas U-shape with area 2960 mm².

The defect log records closed issues in calculation, geometry, export, integration, and GUI Canvas editing. Resolved issues include implementation of previously missing warping/shear-center fields, ANSYS centroid-offset handling, empty stress-point export handling, merge conflict resolution for FEM export work, and Canvas endpoint dragging.

The GitHub Actions workflow builds and tests the no-GUI path on Ubuntu and macOS, and includes a Windows core job. Each job configures with SPT_QT_VERSION=OFF, builds, runs CTest, and runs the examples.

13. Local Verification Evidence

The following commands were run locally for this report in a safe no-GUI configuration:

```
cmake -S . -B build-final-report -DCMAKE_BUILD_TYPE=Release -DSPT_QT_VERSION=OFF
cmake --build build-final-report --config Release
ctest --test-dir build-final-report --output-on-failure
./build-final-report/Library/bin/example1_parametric
./build-final-report/Library/bin/example2_canvas
./build-final-report/Library/bin/example3_dll_batch
git diff --check
```

Observed results:

- CMake configured successfully with AppleClang.
- Qt was intentionally disabled for the verification build.
- SectionPropertyCore, SectionPropertyApi, examples, and SectionPropertyTests built successfully.
- CTest executed the aggregated core/API regression target successfully.
- example1_parametric printed H-section area 6520 mm², C_w = 3.00833e+10 mm⁶, and shear center (50, 125) mm.
- example2_canvas printed Canvas area 2960 mm² and centroid (50, 60) mm.
- example3_dll_batch printed successful H-section and pipe-section batch API results.
- git diff --check reported no whitespace errors.

This verification confirms the local no-GUI core/API build and example workflows. It does not replace the separately documented Windows Qt deployment checklist or solver-side FEM acceptance.

14. User Documentation and Usage Guide

The user-facing workflow is documented by the project and can be summarized as follows.

For predefined sections, the user selects a section type, enters the required dimensions, applies the calculation, and reviews the property table. H, box, pipe, and crane girder parameter names match the API definitions. Invalid or physically impossible combinations produce diagnostics; users are expected to correct input values and recalculate.

For stress output points, the user reviews the generated points, edits global coordinates if needed, and observes updated principal coordinates. Stress points are output locations for reporting; they are not required to coincide with mesh nodes.

For mesh review, the user opens the FE Mesh tab and adjusts the refinement factor. The generated mesh gives visual feedback and summary data. It is not presented as a solver mesh.

For Canvas sections, the user defines plate centerlines and thicknesses using either the table or graphical drawing tools. The Canvas builder checks for numeric coordinates, positive thickness, nonzero segment length, and unique explicit IDs. When validation succeeds, the Canvas section becomes the active model and the property table, stress points, previews, and mesh update.

For export, the user or API client chooses CSV, JSON, ANSYS, ABAQUS, or Midas Civil output. Export failures generally indicate an invalid or unwritable target path.

Common troubleshooting guidance includes:

- Confirm that all required parameters are present and positive.
- For pipes, confirm that thickness is less than half of the outer diameter.
- For box sections, confirm that outer dimensions leave room for web thicknesses.
- Treat non-reference crane girder warnings as an indication of approximate plate-graph behavior.
- Confirm that export paths are writable.
- If the GUI target is not built, confirm that Qt is installed and SPT_QT_VERSION is not OFF.

15. Developer Documentation and Build Instructions

The development workflow requires CMake 3.16 or later, a C++17 compiler, and Git. Qt Widgets is optional. Qt 5 can be selected with SPT_QT_VERSION=5, Qt 6 with SPT_QT_VERSION=6, auto-discovery with AUTO, and no GUI with OFF.

The portable no-GUI build is:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DSPT_QT_VERSION=OFF
cmake --build build
ctest --test-dir build --output-on-failure
```

This path builds the core static library, shared API, examples, and tests without Qt.

Qt 5 and Qt 6 builds follow the same CMake structure but include a Qt prefix path. When Qt Widgets is detected, CMake builds SectionPropertyGui. When Qt Test is also detected, CMake builds and registers SectionPropertySmokeTest.

On Windows, the documented core/API build uses Visual Studio 2022 and SPT_QT_VERSION=OFF. The GUI deployment flow uses build_windows.bat, which detects supported Qt installations, configures the build, runs CTest, and deploys the GUI executable with windeployqt. The Windows verification checklist also identifies expected artifacts such as SectionPropertyTool.dll, SectionPropertyTool.lib, SectionPropertyCore.lib, SectionPropertyTests.exe, SectionPropertySmokeTest.exe, example executables, and SectionPropertyGui.exe.

The CMake targets are:

Target	Purpose
SectionPropertyCore	Core geometry, calculation, mesh, stress, and export library
SectionPropertyApi	Shared library exported as SectionPropertyTool
SectionPropertyTests	Aggregated core/API test executable
example1_parametric	H-section API example
example2_canvas	Canvas section API example
example3_dll_batch	Batch C API example
SectionPropertyGui	Optional Qt Widgets GUI
SectionPropertySmokeTest	Optional offscreen Qt GUI smoke test

The developer guide also describes extension points. Adding a new section type requires changes to the core SectionType, API enum, API conversion, geometry builder, calculation/mesh/drawing paths,

stress point behavior when needed, tests, and documentation. Adding a new export format requires extending the core and API export enums, routing through the API conversion layer, adding an exporter method, adding integration tests, and documenting acceptance status honestly.

No third-party geometry or meshing dependency is bundled. Future geometry or meshing backends should be isolated behind the existing geometry and mesh modules.

16. Validation Boundaries and Future Work

The project is a functional engineering software deliverable for the documented scope. The remaining items are validation or hardening boundaries:

- Windows Qt deployment: final windeployqt packaging and visual/manual runtime inspection should be completed on an interactive Windows machine with the documented Qt installation.
- FEM solver acceptance: ANSYS, ABAQUS, and Midas Civil writers are implemented and integration-tested for generated content, but generated files should be accepted inside the target solver applications before production solver use.
- Non-reference crane girder geometry: arbitrary crane girder parameter sets use approximate plate-graph behavior until complete construction rules are confirmed or formally accepted.
- DOCX manual package: Documents/User Manual.docx is available, but final formatting and packaging review remains pending.
- Documentation maintenance: older notes such as parts of Documents/DLL_Interface.md and tests/gui_smoke/README.md should be reconciled with the current export and GUI smoke-test implementation.
- Requirements traceability: a formal traceability matrix linking source requirements, code modules, tests, and documentation would improve future review packages.
- Mesh capability: the current mesh is a visualization mesh; a solver-quality meshing backend would be future work if solver mesh export becomes a requirement.

17. Conclusion

Beam Element Section Property Tool is suitable for submission as a functional engineering software deliverable. The project includes a C++17 calculation core, C-compatible shared-library API, optional Qt GUI, examples, automated tests, numerical validation material, build scripts, CI configuration, and user/developer documentation.

The implementation covers the documented section types and workflows: parametric H, box, pipe, crane girder reference behaviour, Canvas thin-walled sections, property calculations, stress output points, coordinate transformations, visualization mesh generation, and multiple export formats. The local no-GUI verification confirms that the core/API build, aggregated CTest target, and example workflows execute successfully in the current environment.

The remaining work is appropriately framed as external validation and future hardening. Manual Windows Qt deployment, solver-side FEM acceptance, DOCX final review, documentation reconciliation, and confirmation of arbitrary crane girder construction rules are important next steps, but they do not change the status of the delivered core/API implementation for the documented project scope.